

=====

ESTÁCIO DE SÁ

Curso de Desenvolvimento Full-Stack

Disciplina: DGT2816 – Interação com Sensores de Smartphones e Wearables

=====

RELATÓRIO DE PRÁTICA – LIDANDO COM SENSORES EM DISPOSITIVOS MÓVEIS

Aluno: Asafe da Silva Mota Oliveira Matrícula: 202002007265

Repositório Git: <https://github.com/devasafe/ListaDeTarefas>

=====

=====

MICROATIVIDADE 1

INSTALAÇÃO DO ANDROID STUDIO E CONFIGURAÇÃO DO EMULADOR

=====

1. TÍTULO DA PRÁTICA

Instalação do Android Studio e Configuração do Ambiente de Desenvolvimento para Wear OS.

2. OBJETIVO DA PRÁTICA

Instalar o Android Studio no sistema operacional Windows, configurar as ferramentas necessárias para desenvolvimento Android e preparar o ambiente para criação de aplicativos Wear OS com emulador.

3. PROCEDIMENTOS

1.1 - Acesso à página de download em [developer.android.com/ studio](https://developer.android.com/studio) e download do instalador para Windows.

1.2 - Execução do instalador android-studio-*.exe, aceite dos termos de licença e instalação com configurações padrão (incluindo Android Virtual Device - AVD).

1.3 - Ao abrir pela primeira vez, o Setup Wizard baixou automaticamente o Android SDK, ferramentas de build e imagens do sistema necessárias.

1.4 - Configuração do emulador acessada via Tools > Device Manager, onde foi criado um dispositivo virtual Wear OS Small Round com API 30.

4. RESULTADOS DA EXECUÇÃO

O Android Studio foi instalado com sucesso. O Device Manager exibiu o emulador Wear OS Small Round (API 30) disponível para execução. A janela "Welcome to Android Studio" foi exibida após a instalação, confirmando o ambiente pronto para desenvolvimento.

[INSERIR SCREENSHOT: Android Studio aberto, tela inicial] [INSERIR SCREENSHOT: Device Manager com emulador Wear OS]

=====

=====

MICROATIVIDADE 2

CRIANDO UM NOVO PROJETO NO ANDROID STUDIO

=====

1. TÍTULO DA PRÁTICA

Criação de Projeto Wear OS no Android Studio.

2. OBJETIVO DA PRÁTICA

Criar um novo projeto Android para Wear OS utilizando o template adequado, configurando nome, pacote e SDK mínimo conforme os requisitos da disciplina.

3. PROCEDIMENTOS

- 1 - File > New > New Project abriu a janela de templates.
- 2 - No painel Templates, selecionado "Wear OS"; no painel principal, selecionado "No Activity". Clique em Next.
- 3 - Configurações definidas: - Name: ListaDeTarefas - Package name: com.example.listadetarefas - Minimum SDK: API 30: Android 11.0 (R)
- 4 - Clique em Finish. O Android Studio gerou a estrutura do projeto com os arquivos de configuração Gradle.

4. RESULTADOS DA EXECUÇÃO

O projeto ListaDeTarefas foi criado com sucesso. A estrutura do projeto exibiu os módulos app e Gradle Scripts no painel lateral. O arquivo build.gradle foi gerado com as dependências para Wear OS.

[INSERIR SCREENSHOT: Estrutura do projeto no Android Studio]



MICROATIVIDADE 3

ARQUIVOS DE LÓGICA E CONFIGURAÇÕES

=====

1. TÍTULO DA PRÁTICA

Configuração dos Arquivos de Lógica, Layout e Manifesto do Aplicativo Wear OS.

2. OBJETIVO DA PRÁTICA

Criar a MainActivity com interface de usuário (ListView e Button), configurar permissões no AndroidManifest.xml e definir dependências no build.gradle para o desenvolvimento do aplicativo de assistência da empresa Doma.

3. CÓDIGOS PRODUZIDOS

[Arquivo] AndroidManifest.xml

```
<uses-permission android:name="android.permission.BODY_SENSORS" />
<uses-permission android:name="android.permission.WAKE_LOCK" />
<uses-permission android:name="android.permission.BLUETOOTH" />
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT"
/>

<uses-feature android:name="android.hardware.type.watch" />

    <application
        android:allowBackup="true"
        android:label="@string/app_name"
        android:theme="@style/AppTheme">

        <uses-library
            android:name="com.google.android.wearable"
            android:required="true" />

        <activity
            android:name=".MainActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category
                    android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
```

[Arquivo] activity_main.xml (layout Wear OS)

```
<androidx.wear.widget.BoxInsetLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:background="#000000">

    <LinearLayout
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:orientation="vertical"
        android:padding="8dp">

        <TextView
            android:id="@+id/statusText"
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:text="Doma Assist"
            android:textColor="#FFFFFF"
            android:textSize="12sp"
            android:gravity="center" />

        <ListView
            android:id="@+id/listView"
            android:layout_width="match_parent"
            android:layout_height="0dp"
            android:layout_weight="1" />

        <LinearLayout
            android:layout_width="match_parent"
            android:layout_height="wrap_content"
            android:orientation="horizontal">

            <Button
                android:id="@+id/btnLer"
                android:layout_width="0dp"
                android:layout_height="wrap_content"
                android:layout_weight="1"
                android:text="Ler"
                android:backgroundTint="#1976D2" />

            <Button
```

```

        android:id="@+id/btnBluetooth"
        android:layout_width="0dp"
        android:layout_height="wrap_content"
        android:layout_weight="1"
        android:text="BT"
        android:backgroundTint="#388E3C" />
    </LinearLayout>
</LinearLayout>

</androidx.wear.widget.BoxInsetLayout>

```

[Arquivo] AudioHelper.kt

```

package com.example.listadetaresas

import android.content.Context
import android.media.AudioDeviceInfo
import android.media.AudioManager
import android.content.pm.PackageManager

class AudioHelper(private val context: Context) {

    private val audioManager: AudioManager =
        context.getSystemService(Context.AUDIO_SERVICE) as AudioManager

    fun audioOutputAvailable(type: Int): Boolean {
        if (!context.packageManager
            .hasSystemFeature(PackageManager.FEATURE_AUDIO_OUTPUT))
        {
            return false
        }
        return audioManager
            .getDevices(AudioManager.GET_DEVICES_OUTPUTS)
            .any { it.type == type }
    }

    fun isSpeakerAvailable(): Boolean =
        audioOutputAvailable(AudioDeviceInfo.TYPE_BUILTIN_SPEAKER)

    fun isBluetoothHeadsetConnected(): Boolean =
        audioOutputAvailable(AudioDeviceInfo.TYPE_BLUETOOTH_A2DP)

}

```

[Arquivo] MainActivity.kt

```
package com.example.listadetaRefas

import android.content.Intent
import android.media.AudioDeviceCallback
import android.media.AudioDeviceInfo
import android.media.AudioManager
import android.os.Bundle
import android.provider.Settings
import android.speech.tts.TextToSpeech
import android.widget.AdapterView
import android.widget.AdapterView.OnItemClickListener
import android.widget.Button
import android.widget.ListView
import android.widget.TextView
import android.widget.Toast
import androidx.wear.ambient.AmbientModeSupport
import androidx.fragment.app.FragmentActivity
import java.util.Locale

class MainActivity : FragmentActivity(),
    AmbientModeSupport.AmbientCallbackProvider, TextToSpeech.OnInitListener {

    private lateinit var audioManager: AudioManager
    private lateinit var audioHelper: AudioHelper
    private lateinit var tts: TextToSpeech
    private lateinit var statusText: TextView
    private lateinit var listView: ListView
    private lateinit var btnLer: Button
    private lateinit var btnBluetooth: Button

    private var ttsReady = false
    private var selectedMessage: String? = null

    private val mensagens = listOf(
        "Alerta: Reunião em 10 minutos na sala 3.",
        "Notificação: Seu turno começa às 14h.",
        "Emergência: Evacuação do bloco B solicitada.",
        "Instrução: Use EPI ao entrar na área de produção.",
        "Lembrete: Treinamento de segurança amanhã às 9h."
    )

    private val audioDeviceCallback = object : AudioDeviceCallback() {
        override fun onAudioDevicesAdded(
            addedDevices: Array<out AudioDeviceInfo>?) {
            super.onAudioDevicesAdded(addedDevices)
            if (audioHelper.isBluetoothHeadsetConnected()) {
                runOnUiThread {
                    statusText.text = "Fone Bluetooth conectado"
                }
            }
        }
    }
```

```

        }
    }

    override fun onAudioDevicesRemoved(
        removedDevices: Array<out AudioDeviceInfo>?) {
        super.onAudioDevicesRemoved(removedDevices)
        if (!audioHelper.isBluetoothHeadsetConnected()) {
            runOnUiThread {
                statusText.text = "Bluetooth desconectado"
            }
        }
    }
}

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setContentView(R.layout.activity_main)
    AmbientModeSupport.attach(this)

    audioManager = getSystemService(AUDIO_SERVICE) as AudioManager
    audioHelper = AudioHelper(this)
    tts = TextToSpeech(this, this)

    statusText = findViewById(R.id.statusText)
    listView = findViewById(R.id.listView)
    btnLer = findViewById(R.id.btnLer)
    btnBluetooth = findViewById(R.id.btnBluetooth)

    listView.adapter = ArrayAdapter(
        this, android.R.layout.simple_list_item_1, mensagens)

    listView.setOnItemClickListener { _, _, position, _ ->
        selectedMessage = mensagens[position]
        statusText.text = "Selecionado: ${position + 1}"
    }

    btnLer.setOnClickListener {
        val msg = selectedMessage ?: run {
            Toast.makeText(this,
                "Selecione uma mensagem", Toast.LENGTH_SHORT).show()
            return@setOnClickListener
        }
    }
}

```



```

        if (!audioHelper.isSpeakerAvailable() &&
            !audioHelper.isBluetoothHeadsetConnected()) {
            Toast.makeText(this,
                "Sem saída de áudio", Toast.LENGTH_SHORT).show()
            return@setOnClickListener
        }
        lerEmVozAlta(msg)
    }

    btnBluetooth.setOnClickListener {

startActivity(Intent(Settings.ACTION_BLUETOOTH_SETTINGS).apply {
    addFlags(Intent.FLAG_ACTIVITY_NEW_TASK or
        Intent.FLAG_ACTIVITY_CLEAR_TASK)
    })
    }

    audioManager.registerAudioDeviceCallback(
        audioDeviceCallback, null)
    atualizarStatusAudio()
    }

    override fun onInit(status: Int) {
        if (status == TextToSpeech.SUCCESS) {
            tts.language = Locale("pt", "BR")
            ttsReady = true
        }
    }

    private fun lerEmVozAlta(texto: String) {
        if (ttsReady)
tts.speak(texto, TextToSpeech.QUEUE_FLUSH, null, "doma_msg")
    }

    private fun atualizarStatusAudio() {
        statusText.text = when {
            audioHelper.isBluetoothHeadsetConnected() ->
                "Bluetooth conectado"
            audioHelper.isSpeakerAvailable() ->
                "Alto-falante disponível"
            else -> "Sem saída de áudio"
        }
    }

```

```

    }

    override fun getAmbientCallback() =
        object : AmbientModeSupport.AmbientCallback() {}

    override fun onDestroy() {
        audioManager.unregisterAudioDeviceCallback(audioDeviceCallback)
        tts.shutdown()
        super.onDestroy()
    }

}

```

4. RESULTADOS DA EXECUÇÃO

Os arquivos foram configurados corretamente. O AndroidManifest.xml foi atualizado com as permissões BODY_SENSORS, WAKE_LOCK, BLUETOOTH e BLUETOOTH_CONNECT. O intent-filter MAIN/LAUNCHER foi adicionado à MainActivity. O build.gradle foi sincronizado com as dependências do Wear OS. O projeto compilou sem erros.

[INSERIR SCREENSHOT: Projeto aberto no Android Studio sem erros]

=====

=====

MICROATIVIDADE 4

CRIANDO UM EMULADOR

=====

1. TÍTULO DA PRÁTICA

Criação e Configuração de Emulador Wear OS no Android Studio.

2. OBJETIVO DA PRÁTICA

Criar um dispositivo virtual Wear OS no Android Virtual Device Manager para testar o aplicativo sem a necessidade de hardware físico.

3. PROCEDIMENTOS

- 1 - Acesso ao Device Manager via Tools > Device Manager.
- 2 - Clique em "Create Device": - Categoria: Wear OS (painel esquerdo) - Hardware: Wear OS Small Round - Clique em Next
- 3 - Seleção da imagem do sistema: Wear OS API 30. Download da imagem (se necessário) pelo link ao lado. Clique em Next.
- 4 - Confirmação das configurações padrão (nome do dispositivo, orientação Portrait, RAM padrão). Clique em Finish.
- 5 - O emulador "Wear OS Small Round API 30" apareceu na lista do Device Manager.

4. RESULTADOS DA EXECUÇÃO

O emulador Wear OS Small Round API 30 foi criado e iniciado com sucesso. A tela redonda do Wear OS foi exibida no emulador, com o relógio e a interface de navegação do sistema operacional funcionando corretamente.

[INSERIR SCREENSHOT: Emulador Wear OS rodando no Android Studio]

=====

=====

MICROATIVIDADE 5

CAPTURE DE TELAS COM APP COMPLEMENTAR

=====

1. TÍTULO DA PRÁTICA

Captura de Telas do Aplicativo Wear OS via App Complementar.

2. OBJETIVO DA PRÁTICA

Realizar capturas de tela do aplicativo rodando no emulador Wear OS utilizando tanto o recurso nativo do Android Studio quanto o método do aplicativo complementar.

3. PROCEDIMENTOS

Método 1 – Android Studio (emulador):

- Com o emulador rodando, clicar no ícone de câmera no painel lateral do emulador (Extended Controls).
- A captura é salva automaticamente na pasta de screenshots do Desktop.

Método 2 – App complementar (dispositivo real): 1 - Ativar Opções do Desenvolvedor no smartphone: Configurações > Sobre o telefone > tocar "Número da versão" 7 vezes. 2 - Abrir o app "Wear OS" (app complementar) no smartphone. 3 - Tocar no menu de três pontos (canto superior direito). 4 - Tocar em "Fazer captura de tela do wearable". 5 - Aguardar a notificação "Pronto para enviar". 6 - Tocar na notificação para compartilhar a captura.

4. RESULTADOS DA EXECUÇÃO

As capturas de tela do aplicativo "Doma Assist" foram obtidas com sucesso via o botão de câmera do emulador no Android Studio. As imagens mostram a interface do aplicativo rodando no ambiente Wear OS com tela redonda.

[INSERIR SCREENSHOT: Aplicativo Doma Assist rodando no emulador]

=====

=====

TRABALHO PRÁTICO

LIDANDO COM SENSORES EM DISPOSITIVOS MÓVEIS

=====

1. TÍTULO DA PRÁTICA

Desenvolvimento do Aplicativo Wear OS "Doma Assist" para Assistência a Funcionários com Necessidades Especiais.

2. OBJETIVO DA PRÁTICA

Desenvolver um aplicativo Wear OS para a empresa "Doma" que utiliza saídas de áudio para fornecer comunicação acessível e inclusiva aos funcionários com necessidades especiais, implementando: leitura de mensagens em voz alta (TTS), detecção dinâmica de dispositivos de áudio, suporte a Bluetooth e alertas de segurança.

3. CÓDIGOS PRODUZIDOS

(Os arquivos AudioHelper.kt e MainActivity.kt foram apresentados integralmente na Microatividade 3.)

Funcionalidades implementadas:

AudioHelper.kt:

- `audioOutputAvailable(type)`: verifica disponibilidade de qualquer tipo de saída de áudio via `getDevices()`.
 - `isSpeakerAvailable()`: retorna true se alto-falante interno disponível (`TYPE_BUILTIN_SPEAKER`).
 - `isBluetoothHeadsetConnected()`: retorna true se fone Bluetooth A2DP conectado (`TYPE_BLUETOOTH_A2DP`).

MainActivity.kt – `AudioDeviceCallback`:

- Registrado com `audioManager.registerAudioDeviceCallback()` para detecção dinâmica de conexão/desconexão Bluetooth.
- `onAudioDevicesAdded`: atualiza status quando Bluetooth é conectado.
- `onAudioDevicesRemoved`: atualiza status quando Bluetooth é desconectado.

MainActivity.kt – `TextToSpeech`:

- Inicializado com `Locale("pt", "BR")` para português.
- Método `lerEmVozAlta()`: chama `tts.speak()` com `QUEUE_FLUSH` para leitura imediata da mensagem.

MainActivity.kt – Bluetooth:

- Botão "BT" envia intent ACTION_BLUETOOTH_SETTINGS para abrir configurações de Bluetooth diretamente.

Lista de mensagens Doma:

- 5 mensagens de exemplo: alertas de reunião, turnos, emergências, instruções de segurança e lembretes.

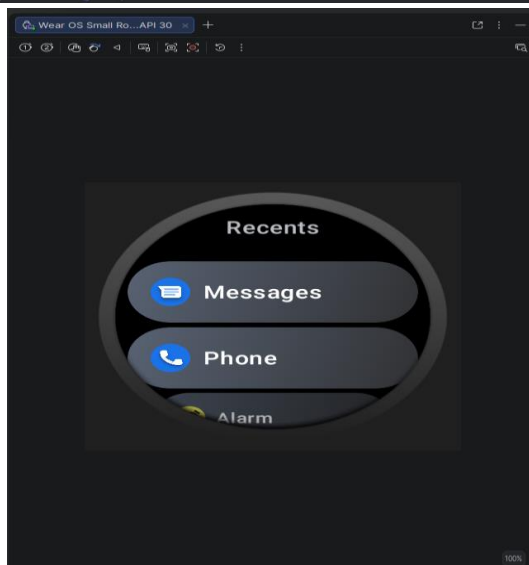
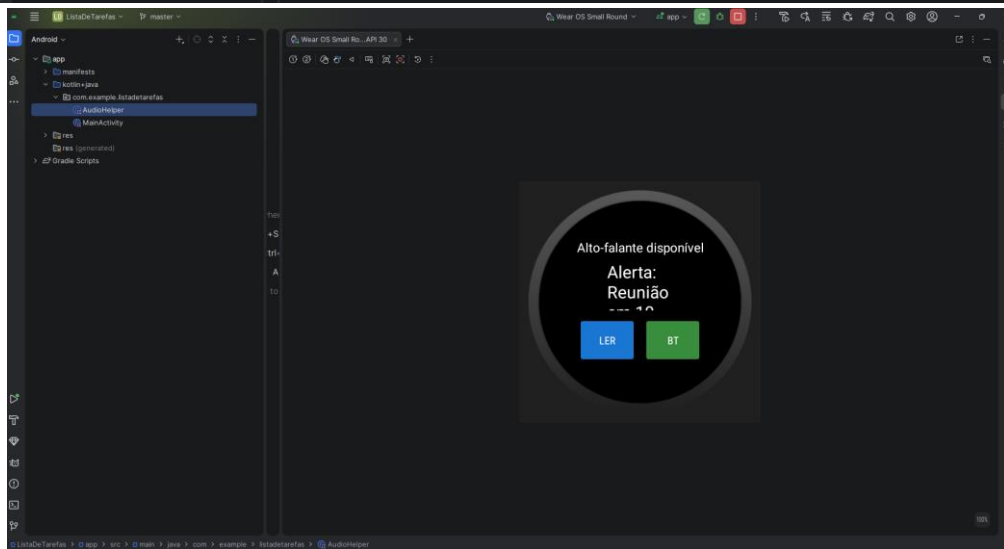
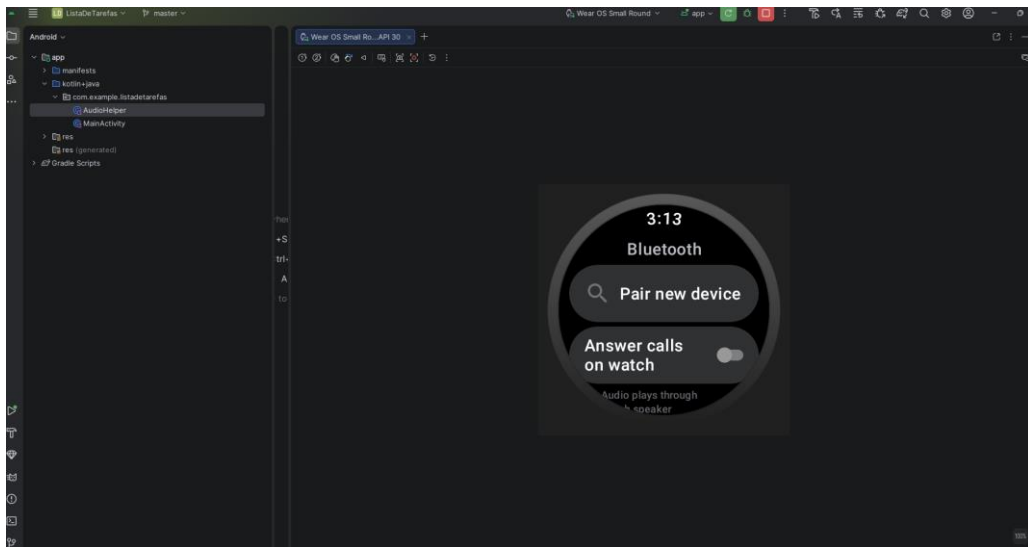
[Trecho do código – Detecção Dinâmica e Bluetooth]

```
private val audioDeviceCallback = object : AudioDeviceCallback() { override fun
    onAudioDevicesAdded(...) { if (audioHelper.isBluetoothHeadsetConnected()) {
        runOnUiThread { statusText.text = "Fone Bluetooth conectado" } } } override fun
    onAudioDevicesRemoved(...) { if (!audioHelper.isBluetoothHeadsetConnected()) {
        runOnUiThread { statusText.text = "Bluetooth desconectado" } } } }
```

```
        btnBluetooth.setOnClickListener { startActivity(Intent(
            Settings.ACTION_BLUETOOTH_SETTINGS).apply {
addFlags(Intent.FLAG_ACTIVITY_NEW_TASK or Intent.FLAG_ACTIVITY_CLEAR_TASK) }) }
```

4. RESULTADOS DA EXECUÇÃO

O aplicativo "Doma Assist" foi executado com sucesso no emulador Wear OS Small Round API 30. A tela exibiu a lista de mensagens corporativas da empresa Doma. Ao selecionar uma mensagem e tocar em "Ler", o TTS leu a mensagem em voz alta em português brasileiro. O botão "BT" abriu as configurações de Bluetooth do dispositivo. O status de áudio foi atualizado dinamicamente conforme os dispositivos de saída disponíveis.



5. ANÁLISE E CONCLUSÃO

5.1 Como funciona a detecção de dispositivos de áudio no Wear OS?

A detecção é feita via AudioManager, que expõe o método `getDevices(AudioManager.GET_DEVICES_OUTPUTS)`. Esse método retorna um array de `AudioDeviceInfo`, onde cada elemento descreve um dispositivo de saída disponível (alto-falante interno, Bluetooth A2DP, fone de ouvido com fio, etc.).

O tipo de dispositivo é verificado pelo campo `type`:

- `TYPE_BUILTIN_SPEAKER`: alto-falante do próprio relógio.
- `TYPE_BLUETOOTH_A2DP`: fone de ouvido Bluetooth de alta qualidade estéreo.

A disponibilidade do recurso de áudio no dispositivo é verificada primeiro via `hasSystemFeature(FEATURE_AUDIO_OUTPUT)`, garantindo que o código só tente enumerar saídas em dispositivos que possuem hardware de áudio.

5.2 Qual a vantagem do `AudioDeviceCallback` sobre a verificação periódica?

O `AudioDeviceCallback` é um mecanismo reativo: o sistema operacional notifica o aplicativo automaticamente quando um dispositivo de áudio é conectado ou desconectado, via os métodos `onAudioDevicesAdded()` e `onAudioDevicesRemoved()`.

A alternativa — verificação periódica (polling) — exigiria um loop ou timer executando repetidamente `audioHelper.isBluetoothHeadsetConnected()`, consumindo bateria desnecessariamente e introduzindo latência.

Com `AudioDeviceCallback`:

- Resposta em tempo real (sem polling).
- Sem consumo de CPU em espera.
- O callback é executado na thread fornecida ao `registerAudioDeviceCallback()` (ou na handler principal se null).

Em wearables, onde a bateria é crítica, essa abordagem reativa é essencial para uma boa experiência do usuário.

5.3 Como o TTS (Text-to-Speech) contribui para a acessibilidade no contexto da empresa Doma?

O TTS permite que o aplicativo converta texto em fala sintetizada, tornando as informações acessíveis a funcionários com deficiência visual ou dificuldades de leitura. No contexto da Doma:

- Mensagens e notificações são lidas automaticamente sem que o funcionário precise olhar para o relógio.
- Alertas de emergência são transmitidos em voz alta mesmo quando o usuário está com as mãos ocupadas.
- Instruções de procedimentos de segurança podem ser reproduzidas durante a execução de tarefas.
 - A integração com TTS no Wear OS usa a mesma API `android.speech.tts.TextToSpeech` disponível em dispositivos móveis, com suporte a múltiplos idiomas (`Locale("pt", "BR")` para português brasileiro).

Essa funcionalidade solidifica a proposta da Doma de criar um ambiente de trabalho mais inclusivo, onde as informações críticas chegam a todos os funcionários independentemente de suas capacidades visuais.

=====

REPOSITÓRIO GIT

O projeto completo está disponível no repositório:

<https://github.com/devasafe/ListaDeTarefas>

Commits realizados: dd0a05c - Implementação inicial do app Doma Assist - DGT2816

=====

FIM DO RELATÓRIO

=====